WEEK 1 · LESSON 4 OF 24 · HANDOUT

# Making Decisions and Writing Functions

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=q0bQrGhdPm4 — open the lesson online to watch inside the course.

**Your goal in this lesson:** Use `if`, `elif` and `else` to make decisions in code, and write your own functions to reuse logic.

## Comparisons give True or False

```
print(7500 > 5000)      # True
print(800 == 850)       # False  (== checks equality)
print("Huye" != "Musanze")   # True  (!= means "not equal")
```

Combine conditions with `and`, `or` and `not`:

```
amount = 60000
is_member = True
print(amount > 50000 and is_member)   # True
```

## if, elif, else

```
amount = 60000

if amount >= 100000:
    discount = 0.10
elif amount >= 50000:
    discount = 0.05
else:
    discount = 0

print(f"Discount: {discount:.0%}")
```

Python checks the conditions **from top to bottom** and runs only the **first** block that is True. `else` catches everything left. `{discount:.0%}` shows 0.05 as `5%`.

Decisions inside loops are very common:

```
marks = [72, 45, 88, 39, 61]
for m in marks:
    if m >= 50:
        print(m, "pass")
    else:
        print(m, "fail")
```

## Functions: write once, use many times

A **function** is a named block of code that takes **inputs** (parameters) and **returns** an output.

```
def price_with_vat(price, rate=0.18):
    # Return the price including VAT.
    return round(price * (1 + rate))

print(price_with_vat(7500))          # 8850
print(price_with_vat(1000, 0.0))     # 1000 (VAT-exempt item)
```

- `def` starts the function; the inputs go in brackets.
- `rate=0.18` is a **default value**, used when you do not give one.
- `return` sends the answer back. Without `return`, a function gives `None`.

A function that makes a decision:

```
def grade(mark):
    if mark >= 80:
        return "A"
    elif mark >= 70:
        return "B"
    elif mark >= 60:
        return "C"
    elif mark >= 50:
        return "D"
    return "F"

for m in [91, 74, 55, 30]:
    print(m, grade(m))
```

## Why functions matter

- **No copy-paste mistakes** — fix the logic once and it is fixed everywhere.
- **Readable code** — `price_with_vat(7500)` explains itself.
- **Testable** — you can check a small function on its own.

In AI, almost everything is a function: a model is a function from features to a prediction; a loss is a function that measures error. In Week 4 you will write both yourself.

## A simple "model" as a function

```
def loan_decision(monthly_sales, late_payments):
    if late_payments >= 3:
        return "refuse"
    if monthly_sales >= 150000:
        return "approve"
    return "review by officer"

print(loan_decision(180000, 0))
print(loan_decision(60000, 1))
print(loan_decision(250000, 4))
```

This is a hand-written **rule-based** model. Machine learning would *learn* the thresholds from past data instead of us choosing them.

## Key terms

- **Condition** — an expression that is True or False.

- **if / elif / else** — choose which block of code runs.
- **Function** — a named, reusable block of code: `def name(inputs):`.
- **Parameter** — an input name in the function definition.
- **Return value** — the result a function sends back.
- **Default value** — a parameter value used when none is given.

**Remember this:** `if/elif/else` lets code make decisions; functions package logic so you can reuse, read and test it. A model, in the end, is a function.

Name: Date:

## Activity

### Task 1

Write `bmi(weight_kg, height_m)` that returns weight ÷ height². Test it with two people.

### Task 2

Write `bmi_category(value)` that returns "under", "normal", "over" or "obese" (use 18.5, 25 and 30 as limits).

### Task 3

Write `transport_fare(km, raining=False)` that returns 400 + 230 × km, plus 300 if it is raining. Test it for 5 km dry and 5 km in rain.

### Task 4

Loop over a list of 5 distances and print the fare for each.

## Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does `==` do?
   - A. Stores a value
   - B. Adds two numbers
   - C. Checks whether two values are equal
   - D. Starts a function
2. With amount = 60000, which block runs? if amount >= 100000 … elif amount >= 50000 … else …
   - A. The if block
   - B. The elif block
   - C. The else block
   - D. All of them
3. What keyword starts a function definition?
   - A. function
   - B. def
   - C. func
   - D. return
4. What does a function give back if it has no `return`?
   - A. An error
   - B. 0
   - C. None
   - D. The last line
5. In `def price_with_vat(price, rate=0.18):`, what is `rate=0.18`?
   - A. An error
   - B. A loop
   - C. A comment
   - D. A default value used when no rate is given
6. Why are functions useful?
   - A. They delete data
   - B. They are only for games
   - C. They make code slower
   - D. They let you reuse, read and test logic without copy-paste

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.